Simulação de Detectores com Geant4 e Geradores de Eventos

Métodos Computacionais em Física de Partículas

Curso de Física de Altas Energias

Instituto de Física – UNICAMP

22 de outubro de 2026

O Desafio da Simulação em Física de Partículas

Simulação em Dois Estágios

Na Física de Altas Energias (HEP), a simulação do experimento é dividida em:

1. **Geração de Eventos (Event Generation):** Descreve a reação fundamental entre partículas, produzindo quadrivetores de momento no estado final (ex: Pythia8, MadGraph5, HERWIG).
2. **Simulação de Detector (Detector Simulation):** Transporta as partículas através da geometria do experimento e simula suas interações físicas com a matéria.

O Método de Monte Carlo

A passagem de partículas pela matéria envolve processos estocásticos (ionização, espalhamento Coulombiano múltiplo, cascatas eletromagnéticas e hadrônicas). Por isso, o transporte é feito via **Monte Carlo**.

O Toolkit Geant4

- **O que é o Geant4?**
  - Pacote livre em C++ desenvolvido pela colaboração internacional CERN/Geant4.
  - Simula com alta precisão a passagem de partículas pela matéria.
- **Funcionalidades Incluídas:**
  - Descrição de geometria complexa e materiais.
  - Modelagem de processos eletromagnéticos e hadrônicos.
  - Rastreamento (tracking) em campos magnéticos e elétricos.
  - Resposta de detectores sensíveis e geração de hits.
  - Visualização 3D e interface com análise de dados (ROOT).

Aplicações

- Colisores de Partículas (ATLAS, CMS, ALICE, LHCb)
- Experimentos de chuveiros atmosféricos
- Física de Neutrinos (DUNE, LBNL) e Matéria Escura
- Física Médica (Radioterapia e PET)
- Engenharia Espacial e Satélites

Fluxo de Execução e o `main()`

O Papel do `G4RunManager`

O Geant4 é um toolkit, ou seja, não possui um programa principal pré-compilado. O usuário deve escrever a função `main()` instanciando o `G4RunManager`.

Responsabilidades do RunManager

- Controla o fluxo da simulação.
- Gerencia o loop de eventos dentro de um run.
- Garante a inicialização das tabelas de seção de choque.

Classes Obrigatórias do Usuário

1. `G4VUserDetectorConstruction`
2. `G4VUserPhysicsList`
3. `G4VUserPrimaryGeneratorAction`

Estrutura Típica de um `main()`

```cpp
#include "G4RunManagerFactory.hh"
#include "G4UImanager.hh"
#include "G4VisExecutive.hh"
#include "G4UIExecutive.hh"

#include "MyDetectorConstruction.hh"
#include "MyPhysicsList.hh"
#include "MyPrimaryGeneratorAction.hh"

int main(int argc, char** argv) {
    // 1. Instancia o RunManager (suporte a Multi-Threading)
    auto* runManager = G4RunManagerFactory::CreateRunManager(G4RunManagerType::Default);

    // 2. Registra as classes obrigatorias de inicializacao do usuario
    runManager->SetUserInitialization(new MyDetectorConstruction());
    runManager->SetUserInitialization(new MyPhysicsList());
    runManager->SetUserAction(new MyPrimaryGeneratorAction());

    // 3. Inicializa o ambiente Geant4
    runManager->Initialize();

    // 4. Executa a simulacao para N eventos
    runManager->BeamOn(100);

    delete runManager;
    return 0;
}
```

Hierarquia de Execução no Geant4



- **Run:** Conjunto de eventos compartilhando a mesma geometria e lista de física.
- **Event:** Simulação de um único feixe/colisão e todas as partículas geradas.
- **Track:** Estado de uma partícula em movimento (posição, momento, energia, carga).
- **Step:** O menor segmento de trajetória entre duas interações físicas consecutivas.

Construção da Geometria em 3 Passos

Para definir qualquer elemento do detector no Geant4, seguimos três passos fundamentais:

1. Solid (`G4VSolid`)

Define a forma geométrica pura e dimensão espacial (sem material).

- `G4Box`
- `G4Tubs`
- `G4Sphere`
- `G4Cons`

2. Logical Volume

Associa o `G4VSolid` a um **Material** e define atributos (sensibilidade, campo E/B).

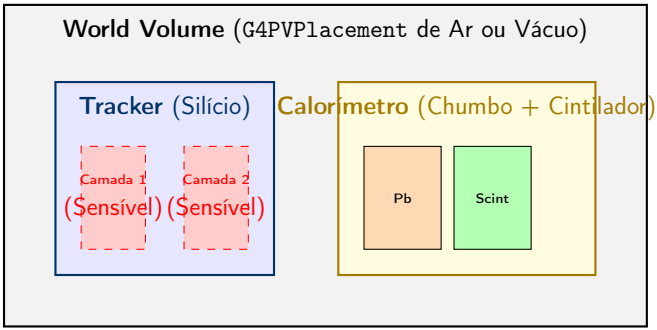
- `G4LogicalVolume`

3. Physical Volume

Posiciona a cópia do Logical Volume no espaço tridimensional.

- `G4PVPlacement`
- `G4PVReplica`

Diagrama da Hierarquia de Volumes (World & Daughter Volumes)



Definição de Materiais e Unidades

Sistema de Unidades Explícito

No Geant4, toda grandeza física deve ser multiplicada pela sua unidade `G4SystemOfUnits.hh`). Para obter o valor numérico em determinada unidade, divida por ela!

```
#include "G4NistManager.hh"
#include "G4Material.hh"
#include "G4SystemOfUnits.hh"
// 1. Usando a base de dados do NIST (Forma recomendada)
G4NistManager* nist = G4NistManager::Instance();
G4Material* air    = nist->FindOrBuildMaterial("G4_AIR");
G4Material* lead   = nist->FindOrBuildMaterial("G4_Pb");
G4Material* si     = nist->FindOrBuildMaterial("G4_Si");
G4Material* poly   = nist->FindOrBuildMaterial("G4_POLYSTYRENE");
// 2. Construindo um material customizado a partir dos elementos
G4double density = 1.03 * g/cm3;
G4Material* scintillator = new G4Material("Scintillator", density, 2);
G4Element* C = nist->FindOrBuildElement("C");
G4Element* H = nist->FindOrBuildElement("H");
scintillator->AddElement(C, 9);
scintillator->AddElement(H, 10);
// Exemplo de calculo: dividindo pela unidade para imprimir
G4double length = 50.0 * cm;
G4cout << "Comprimento em mm: " << length / mm << " mm" << G4endl;
```

Modelagem de Processos Físicos

O Geant4 cobre interações desde 250 eV até a escala de TeV:

Eletromagnéticos (EM)

- **Fótons:** Efeito Fotoelétrico, Espalhamento Compton, Produção de Pares (e^+e^-), Rayleigh.
- **Elétrons/Pósitrons:** Ionização (δ-rays), Bremsstrahlung, Aniquilação de pósitrons.
- **Múons e Hadrões:** Ionização, Par-Produção direta.

Hadrônicos e Ópticos

- **Hadrônicos:** Espalhamento elástico/inelástico, captura de nêutrons, de-excitação e fragmentação nuclear.
- **Ópticos:** Radiação de Cherenkov, Cintilação, Absorção óptica e deslocamento de comprimento de onda (WLS).

Amostragem Monte Carlo do Tamanho do Passo (*Step*)

A probabilidade de uma partícula percorrer uma distância L sem sofrer a interação física i é dada por:

$$P_i(L) = 1 - \exp\left(-\frac{L}{\lambda_i}\right) \tag{1}$$

onde $\lambda_i = \frac{1}{n\sigma_i}$ é o **percurso livre médio** (*mean free path*).

Algoritmo de Transporte a Cada Passo

1. Para cada processo ativo i, sorteia-se uma distância de voo livre $L_i = -\lambda_i \ln(\xi)$, com $\xi \in (0, 1]$.
2. Seleciona-se o menor comprimento de passo $L_{\text{min}} = \min_i(L_i, L_{\text{geom}})$.
3. Transporta-se a partícula por L_{min} e executa-se a física do processo vencedor.

Listas de Física Pré-definidas (*Reference Physics Lists*)

O Geant4 disponibiliza *Physics Lists* validadas pelo CERN para diferentes aplicações:

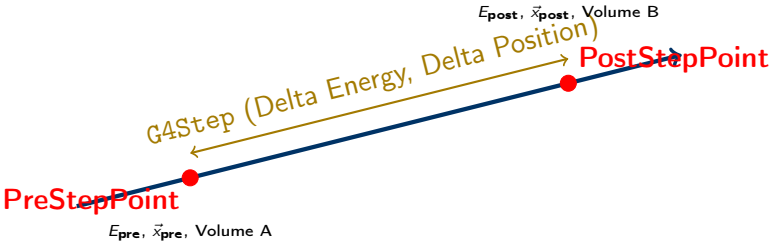
- `FTFP_BERT`: Padrão em Física de Altas Energias (LHC). Usa o modelo Fritiof string para altas energias (> 3 GeV) e Bertini Cascade para energias menores.
- `QGSP_BERT`: Modelo Quark-Gluon String para energias acima de ~ 12 GeV.
- `G4EmStandardPhysics_option4`: Alta precisão para física eletromagnética (usado em física médica e micro-dosimetria).

```
#include "FTFP_BERT.hh"

int main() {
    auto* runManager = G4RunManagerFactory::CreateRunManager();

    // Instanciando a Physics List pre-definida FTFP_BERT
    G4VModularPhysicsList* physicsList = new FTFP_BERT();
    runManager->SetUserInitialization(physicsList);
    ...
}
```

Anatomia do Passo: G4Step e G4StepPoint



Extração de Informação Física

No G4Step, podemos acessar:

- `step->GetTotalEnergyDeposit()`: Energia depositada no passo.
- `step->GetStepLength()`: Comprimento geométrico do passo.
- `track->GetDefinition()->GetParticleName()`: Nome da partícula.

Detector Sensível (G4VSensitiveDetector) e Hits

Para coletar os depósitos de energia do detector e transformar em sinais medíveis:

```cpp
#include "G4VSensitiveDetector.hh"

class MyCalorimeterSD : public G4VSensitiveDetector {
public:
    MyCalorimeterSD(G4String name) : G4VSensitiveDetector(name) {}

    virtual G4bool ProcessHits(G4Step* aStep, G4TouchableHistory* ROhist) override {
        G4double edep = aStep->GetTotalEnergyDeposit();
        if (edep == 0.) return false;

        G4ThreeVector pos = aStep->GetPreStepPoint()->GetPosition();
        G4int copyNo = aStep->GetPreStepPoint()->GetTouchable()->GetCopyNumber();

        // Salva a informacao de Hit ou acumula energia por canal
        G4cout << "Hit no Canal " << copyNo << " | Edep = " << edep/MeV << " MeV" << G4endl;
        return true;
    }
};
```

Geradores de Eventos (Pythia8 e MadGraph5)



Cadeia

- Partículas de meia-vida curta (ex: Z^0, $W^{\pm}$, Higgs H, Quark Top t) devem ser decaindo no gerador (Pythia8 / MadGraph5).
- O Geant4 recebe apenas as partículas com distâncias de voo macroscópicas ($e^{\pm}, \gamma, \mu^{\pm}, \pi^{\pm}, K, p, n$).

Integração Pythia8 → Geant4 C++

```
#include "Pythia8/Pythia.h"
#include "G4VUserPrimaryGeneratorAction.hh"
#include "G4Event.hh"
#include "G4PrimaryVertex.hh"
#include "G4PrimaryParticle.hh"
class PythiaPrimaryGenerator : public G4VUserPrimaryGeneratorAction {
private:
    Pythia8::Pythia pythia;
public:
    PythiaPrimaryGenerator() {
        pythia.readString("Beams:idA = 2212"); // Protons
        pythia.readString("Beams:idB = 2212");
        pythia.readString("Beams:eCM = 13000."); // 13 TeV
        pythia.readString("WeakSingleBoson:ffbar2gmZ = on"); // pp -> Z/gamma*
        pythia.init();
    }
    virtual void GeneratePrimaries(G4Event* anEvent) override {
        pythia.next(); // Gera 1 colisao pp
        G4PrimaryVertex* vertex = new G4PrimaryVertex(0.*cm, 0.*cm, 0.*cm, 0.*ns);
        for (int i = 0; i < pythia.event.size(); ++i) {
            if (pythia.event[i].isFinal()) { // Apenas particulas finais estaveis!
                G4PrimaryParticle* particle = new G4PrimaryParticle(
                    pythia.event[i].id(),pythia.event[i].px()*GeV,
                    pythia.event[i].py()*GeV,pythia.event[i].pz()*GeV);
                vertex->SetPrimary(particle);
            }
        }
        anEvent->AddPrimaryVertex(vertex);
    }
};
```

Gerenciadores de UI e Vis: `G4UIExecutive` e `G4VisExecutive`

Interface de Usuário

O `G4UIExecutive` seleciona a melhor interface disponível para o ambiente (Qt GUI, Terminal CINT/Tcsh, etc.).

Driver de Visualização

O `G4VisExecutive` inicializa os motores gráficos 3D (OpenGL, RayTracer, VRML, DAWN).

Modos de Execução e Solução Definitiva MT/Qt

- **Modo Interativo (GUI):** Usa `G4RunManagerType::SerialOnly` → Executa o loop e a renderização OpenGL na mesma thread do Qt (Main Thread), eliminando totalmente o erro de *QOpenGLContext*.
- **Modo Batch (Massa):** Usa `G4RunManagerType::Default` → Ativa o Multi-Threading paralelo em clusters de computação sem abrir janela gráfica.

Comandos Interativos Essenciais (vis.mac)

```
# 1. Abre o visualizador OpenGL
/vis/open OGL 600x600-0+0

# 2. Desenha a geometria do detector
/vis/drawVolume

# 3. Configura o ponto de vista e estilo de renderizacao
/vis/viewer/set/viewpointVector 1 1 1
/vis/viewer/set/style surface
/vis/viewer/set/autoRefresh true

# 4. Adiciona trajetorias de particulas e pontos de acumulo de energia
/vis/scene/add/trajectories smooth
/vis/scene/add/hits

# 5. Define cores por tipo de particula (e- vermelho, e+ azul, gamma verde, n amarelo)
/vis/modeling/trajectories/create/drawByParticleID

# 6. Dispara 5 eventos de teste com trajetorias visiveis
/run/beamOn 5
```

Exportação de Dados com G4AnalysisManager

O Geant4 inclui uma interface nativa unificada para exportar Histogramas e Ntuples nos formatos ROOT, CSV ou XML:

```cpp
#include "G4AnalysisManager.hh"

// No RunAction::BeginOfRunAction
auto analysisManager = G4AnalysisManager::Instance();
analysisManager->OpenFile("output_data.root");

// Cria um Histograma 1D (id, titulo, nbins, xmin, xmax)
analysisManager->CreateH1("Edep", "Deposicao de Energia em MeV", 100, 0.0, 1000.0);

// Cria uma Ntuple (TTree do ROOT)
analysisManager->CreateNtuple("Hits", "Dados de Passos no Detector");
analysisManager->CreateNtupleDColumn("Energy");
analysisManager->CreateNtupleDColumn("X");
analysisManager->FinishNtuple();

// No EventAction::EndOfEventAction
analysisManager->FillH1(0, totalEdep/MeV);
analysisManager->AddNtupleRow();

// No RunAction::EndOfRunAction
analysisManager->Write();
analysisManager->CloseFile();
```

Saída no Terminal: Mensagens de Run e Verbosidade

Controlando o Nível de Detalhe

O Geant4 possui um sistema de *verbosidade* em múltiplos níveis. Use os comandos abaixo para ajustar o volume de saída.

```
# Verbosidade do RunManager (0=silencioso, 1=resumo por run, 2=por evento)
/run/verbose 1
# Verbosidade de rastreamento (0=off, 1=por track, 2=por step)
/tracking/verbose 1
# Verbosidade dos processos fisicos (lista todos os processos registrados)
/process/verbose 0; /process/list
```

Exemplo de Saída Típica de um Run

```
### Run 0 starts.
--> Event 0 starts.       # cada evento = uma colisao/disparo
--> Event 1 starts.
...
 Run Summary
  Number of events processed : 10
  User=0.12s Real=0.14s Sys=0.00s    # tempo de CPU vs. real
```

Interpretando o Rastreamento Passo-a-Passo

Com `/tracking/verbose 1`, o Geant4 imprime a história de cada partícula:

```
**** G4Tracking: Start new track ****
TrackID=1 ParentID=0  ParticleName=e-   KineticEnergy=500 MeV
Step#  X(mm)   Y(mm)   Z(mm)    KinE(MeV)  dE(MeV)  StepLen(mm)  Process
   0   0.000   0.000   0.000    500.000    0.000    0.000        initStep
   1   0.023   0.001   0.234    499.891    0.109    0.234        eBrem
   2   0.041   0.001   0.461    499.734    0.157    0.227        eIoni
   3   0.058   0.002   0.698    499.612    0.122    0.238        eBrem
  ...
  47   0.103   0.009  10.811      0.000  499.612   ...          StopAndKill
```

O que cada coluna significa

- **TrackID/ParentID**: identifica a árvore de partículas. ParentID=0 $\Rightarrow$ primária.
- **KinE**: energia cinética *após* o passo — leia como o elétron foi freando.
- **dE**: energia *depositada localmente* no meio. A soma de dE ao longo de toda a trajetória é a energia total absorvida.
- **StepLen**: comprimento geométrico percorrido neste passo.

Física dos Processos: o que cada nome significa

O campo `Process` na saída de tracking indica **qual interação física encerrou o passo**:

Processos Eletromagnéticos

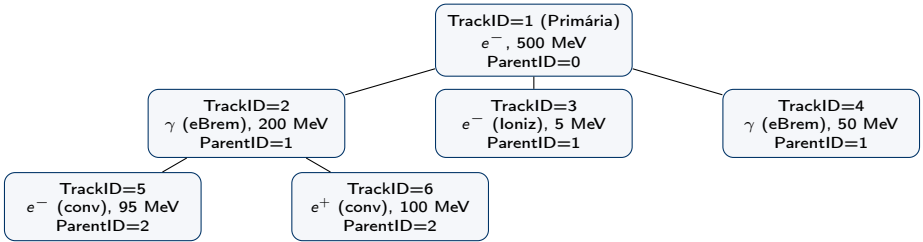
- `eBrem`: Elétron emite fóton de **Bremsstrahlung**. fóton carrega a maior parte da energia para longe.
- `eIoni`: **Ionização**. Elétron arranca elétrons do átomo. dE = energia depositada na matéria.
- `conv`: Fóton converte em par e^+e^- (**produção de pares**).
- `compt`: **Espalhamento Compton**. Fóton perde parte da energia e muda de direção.
- `phot`: **Efeito fotoelétrico**. Fóton é absorvido e eletron é emitido com $E_k = h\nu - \phi$.
- `annihil`: Pósitron se **aniquila** com elétron emitindo 2 fótons de 511 keV.

Processos Hadrônicos e de Transporte

- `hIoni`: **Ionização hadrônica** (próton, píon, kaon...). A perda de energia segue a fórmula de Bethe-Bloch.
- `pi+Inelastic`: **Interação inelástica** do π^+ com um núcleo — inicia chuveiro hadrônico.
- `nCapture`: **Captura de nêutron**. O núcleo absorve o nêutron e emite raios γ.
- `Transportation`: A partícula **cruzou uma fronteira geométrica**. Não é processo físico; indica que mudou de volume lógico.
- `StopAndKill`: A partícula parou (energia cinética = 0) e foi **eliminada da pilha**.

Interpretação da Árvore de Partículas (TrackID/ParentID)

O Geant4 mantém uma **árvore de tracks**: cada processo pode criar partículas secundárias.



Como ler: conservação e chuveiro EM

Um único elétron de 500 MeV em Pb incia um **chuveiro eletromagnético**: eBrem $\rightarrow$ fótons $\rightarrow$ produção de pares $\rightarrow$ novos elétrons $\rightarrow$ mais eBrem... O chuveiro termina quando a energia das partículas cai abaixo da **energia crítica** $E_c \approx 7.4$ MeV no Pb. O número total de partículas cresce exponencialmente antes de saturar: $N_{\max} \propto E_0/E_c$.

Interpretação do Perfil Longitudinal de Deposição de Energia

O perfil $\langle dE/dz \rangle$ por camada do calorímetro revela a **física do chuveiro**:

Chuveiro Eletromagnético (EM)

- Cresce exponencialmente nas primeiras camadas (multiplicação de pares).
- Atinge o máximo na profundidade $t_{\max} \approx \ln(E_0/E_c) - 0.5$ (em unidades de X_0, comprimento de radiação).
- Decai suavemente após $t_{\max}$: fótons e $e^{\pm}$ com $E < E_c$ são absorvidos por ionização ou fotoelétrico.
- **Largura lateral** caracterizada pelo *Raio de Molière* R_M: 90% da energia depositada em um cilindro de raio R_M.

Chuveiro Hadrônico

- Muito mais largo e profundo que o EM: escala com λ_I (livre caminho de interação), não X_0.
- Componente EM (de $\pi^0 \to \gamma\gamma$) + componente hadrônica (prótons, nêutrons recuos nucleares).
- Parte da energia é **invisível**: ligação nuclear, neutrinos de decaimento.
- Resolução de energia intrínsecamente pior: $\sigma/E \sim 50\%/\sqrt{E}$ (hadrônico) vs. $\sim 3\%/\sqrt{E}$ (EM).

Saída em Arquivo: Lendo o CSV do G4AnalysisManager

Quando se usa o formato **CSV**, o Geant4 gera um arquivo por ntuple (um por thread):

```
# Arquivo: calo_output_nt_CaloHits_t0.csv
#class tools::wcsv::ntuple
#column double Edep_MeV  # energia depositada no passo (MeV)
#column int LayerNo      # numero da camada do calorimetro
#column double x_mm      # posicao x do PreStepPoint (mm)
3.142,0,0.00,0.00,5.00
1.987,1,0.00,0.00,15.00
```

```
import pandas as pd, matplotlib.pyplot as plt, glob

# Concatena todos os arquivos de threads (t0, t1, t2...)
frames = [pd.read_csv(f, comment='#', header=None,
          names=['Edep_MeV','Layer','x','y','z'])
          for f in glob.glob('calo_output_nt_CaloHits_t*.csv')]
df = pd.concat(frames, ignore_index=True)

# Perfil longitudinal medio (em unidades de X0 do Pb = 5.6 mm)
X0_mm = 5.6
profile = df.groupby('Layer')['Edep_MeV'].mean()
profile.index = profile.index * 10 / X0_mm  # posicao em X0
profile.plot(xlabel=r'Profundidade [$X_0$]',
             ylabel='<dE/dz> (MeV/camada)',
             title='Perfil Longitudinal do Chuveiro EM')
plt.tight_layout(); plt.savefig('perfil_longitudinal.pdf')
```

Interpretando Histogramas de Energia Depositada

O histograma de E_{dep} por evento contém informação sobre a **resposta do detector**:

```python
import pandas as pd, matplotlib.pyplot as plt, numpy as np
from scipy.optimize import curve_fit

# Le histograma H1 gerado pelo G4AnalysisManager
h = pd.read_csv('calo_output_h1_EdepTotal.csv', comment='#',
                header=None, names=['E_MeV','counts'])
h = h[pd.to_numeric(h['E_MeV'], errors='coerce').notna()].astype(float)

# Ajuste gaussiano para extrair resolucao de energia
def gauss(x, N, mu, sigma):
    return N * np.exp(-0.5*((x-mu)/sigma)**2)

E, n = h['E_MeV'].values, h['counts'].values
popt, _ = curve_fit(gauss, E, n, p0=[n.max(), E[n.argmax()], 20.])
mu, sigma = popt[1], abs(popt[2])
print(f'<E> = {mu:.1f} MeV,  sigma/E = {sigma/mu*100:.1f}%')

plt.bar(E, n, width=(E[1]-E[0])*0.9, alpha=0.6, label='Geant4')
plt.plot(E, gauss(E, *popt), 'r-', label=fr'Gaussiana: $\sigma/E={sigma/mu*100:.1f}\%$')
plt.xlabel('Energia Depositada (MeV)'); plt.legend(); plt.savefig('histo_edep.pdf')
```

Interpretação Física

$\sigma/E = a/\sqrt{E} \oplus b$ onde: a (termo estocástico) $\propto$ flutuação do número de portadores de carga (fótons ou elétrons por MeV); b (ruído constante) $\propto$ não-uniformidade do detector e fuga de energia.